AI Transit Pipeline

Installation Guide

Version 2.0 · 2026-08-12

Applicable to: `fetch_repo.sh` · `scan_pipeline.sh` · `ai_transit.sh` · `generate_excel_report.py` · `selfcheck.py`

Table of Contents

1. Overview

The AI Transit Pipeline is a Bash-based security gateway designed to scan AI-generated code repositories before integration into an enterprise network. The bundle consists of:

File	Role
`fetch_repo.sh`	`Clone/copy the target repository`
`scan_pipeline.sh`	5-layer static analysis engine
`ai_transit.sh`	Main orchestrator (calls fetch + scan)
`generate_excel_report.py`	Excel report generator (openpyxl)

Supported platforms:

- Linux (Ubuntu 22.04 LTS / Debian 12 — **recommended production platform**)
- Windows 11 via WSL2 (development / evaluation only)

2. Prerequisites

Minimum system requirements

Resource	Minimum	Recommended
CPU	2 cores	4 cores
RAM	4 GB	8 GB
Disk	20 GB free	50 GB free
OS	`Ubuntu 22.04 / Debian 12`	Ubuntu 24.04 LTS

Required tools (all platforms)

Tool	Minimum version	Purpose
Bash	5.0	Pipeline runtime
Git	2.30	Repository cloning
Python	3.10	Excel report generation
pip	23.x	Python package manager
curl	7.x	GitHub API size check
jq	1.6	JSON parsing
`zip / unzip`	any	Archive creation
file	any	MIME-type detection
sha256sum	any	Integrity manifests

Optional but strongly recommended tools

Tool	Purpose	Layer
gitleaks	`Secret/credential leak detection`	L1
detect-secrets	Entropy-based secret detection	L1
ClamAV (clamscan)	Antivirus scan	L1
YARA	Custom malware rule matching	L1
Semgrep	`OWASP / CWE / CERT static analysis`	L2
trivy	SCA — CVE in dependencies	L3
pip-audit	Python dependency CVE check	L3
safety	Python dependency advisory check	L3
npm audit	Node.js dependency CVE check	L3
Bandit	Python SAST	L5
ShellCheck	Shell script SAST	L5
hadolint	Dockerfile linter	L5
cppcheck	`C/C++ static analysis`	L5
checkov	`Terraform / IaC security scan`	L5

3. Linux Installation

3.1 System packages

```
sudo apt-get update && sudo apt-get upgrade -y

# Core dependencies
```

```
sudo apt-get install -y \
bash git curl jq zip unzip file coreutils \
python3 python3-pip python3-venv \
build-essential

# Security tools — Layer 1
sudo apt-get install -y clamav clamav-daemon yara

# Security tools — Layer 5
sudo apt-get install -y shellcheck cppcheck
```

3.2 Python packages

```
# Create and activate a dedicated virtual environment (recommended)
python3 -m venv /opt/ai-transit/venv
source /opt/ai-transit/venv/bin/activate

# Install required Python packages
pip install --upgrade pip
pip install openpyxl # Excel report generation
pip install detect-secrets # L1: secret detection
pip install bandit # L5: Python SAST
pip install pip-audit # L3: Python CVE check
pip install safety # L3: Python advisory check
pip install semgrep # L2: OWASP/CWE/CERT analysis
```

> ***Note:*** *If you use a virtual environment, ensure* `activate` *is called before running* `ai_transit.sh`*, or use the full path to the venv's Python:*
>
> *export PATH="/opt/ai-transit/venv/bin:$PATH"*

3.3 gitleaks (L1 — secret scanning)

```
# Download latest release from GitHub
GITLEAKS_VERSION="8.18.4"
curl -sSL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}/gitleaks_${GITL
EAKS_VERSION}_linux_x64.tar.gz" \
-o /tmp/gitleaks.tar.gz
tar -xzf /tmp/gitleaks.tar.gz -C /tmp gitleaks
sudo mv /tmp/gitleaks /usr/local/bin/gitleaks
sudo chmod +x /usr/local/bin/gitleaks
gitleaks version
```

3.4 trivy (L3 — SCA / CVE)

```
# Aqua Security official install script
curl -sfL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh \
| sudo sh -s -- -b /usr/local/bin
trivy --version
```

3.5 hadolint (L5 — Dockerfile)

```
HADOLINT_VERSION="2.12.0"
curl -sSL
"https://github.com/hadolint/hadolint/releases/download/v${HADOLINT_VERSION}/hadolint-Linux-x86_64" \
-o /usr/local/bin/hadolint
sudo chmod +x /usr/local/bin/hadolint
hadolint --version
```

3.6 checkov (L5 — Terraform / IaC)

```
pip install checkov
checkov --version
```

3.7 ClamAV — update virus database

```
sudo systemctl stop clamav-freshclam
sudo freshclam
sudo systemctl start clamav-freshclam
sudo systemctl enable clamav-freshclam
```

3.8 Node.js + npm (for npm audit — L3)

```
# Via NodeSource (LTS)
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
node --version && npm --version
```

4. Windows Installation

> ***Strong recommendation:*** *Use* ***WSL2 (Windows Subsystem for Linux)*** *with Ubuntu 22.04. The pipeline relies on Bash features (associative arrays, process substitution, POSIX tools) that are not reliably available in native Windows environments such as Git Bash or Cygwin.*

4.1 Enable WSL2

Run the following in PowerShell **as Administrator**:

```
# Enable WSL2
wsl --install

# Install Ubuntu 22.04 LTS
wsl --install -d Ubuntu-22.04
```

```
# Set WSL2 as default version
wsl --set-default-version 2

# Restart your machine, then launch Ubuntu from the Start menu
```

After the Ubuntu terminal opens and you have set up your UNIX username and password, follow the **Linux Installation** steps above inside the WSL2 terminal.

4.2 Accessing files from Windows

- WSL2 home directory: `\\wsl$\Ubuntu-22.04\home\<your-user>\`
- Place your pipeline scripts under the WSL2 filesystem (not `/mnt/c/...`) to avoid permission and line-ending issues.

```
# Inside WSL2 terminal
mkdir -p ~/ai-transit
cp /mnt/c/Users/<you>/Downloads/ai-transit-bundle/* ~/ai-transit/
cd ~/ai-transit
chmod +x *.sh
```

4.3 Windows Terminal (recommended)

Install **Windows Terminal** from the Microsoft Store for a better experience. Set the Ubuntu profile as the default.

4.4 Docker Desktop (optional — for trivy)

If you prefer running trivy via Docker instead of installing the binary:

```
# Inside WSL2
docker pull aquasec/trivy:latest
alias trivy='docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
-v $HOME/.cache/trivy:/root/.cache/trivy aquasec/trivy'
```

4.5 Native Windows — not recommended

If WSL2 is not available (e.g., corporate policy blocks Hyper-V), the following partial approach may work using **Git Bash** + **Chocolatey**:

```
# Install Chocolatey
Set-ExecutionPolicy Bypass -Scope Process -Force
[System.Net.ServicePointManager]::SecurityProtocol =
[System.Net.ServicePointManager]::SecurityProtocol -bor 3072
iex ((New-Object System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))

# Install tools
choco install git python3 jq curl zip nodejs -y
pip install openpyxl detect-secrets bandit pip-audit semgrep checkov
```

> ***Limitations of native Windows / Git Bash:***
>
> *- Associative arrays (`declare -A`) require Bash 4+; Git Bash ships Bash 3.x on some versions.*
>
> *- `file --mime-type` may not be available.*

> - `clamav`, `yara`, `gitleaks`, `trivy` *require manual binary placement.*
>
> - *Line endings (CRLF vs LF) can break shell scripts — always convert with* `dos2unix`.

>

> ***For production use, WSL2 is mandatory.***

5. Directory Structure Setup

The pipeline uses the following directory layout (configurable via `WORK_DIR`):

```
/opt/ai-transit/ ← WORK_DIR (default)
■■■ fetch/ ← Cloned repositories (temporary)
■■■ quarantine/ ← Failed repos (chmod 700, isolated)
■■■ approved/ ← (optional) Approved repos mirror
■■■ reports/ ← JSON scan reports
■■■ logs/ ← Pipeline logs
■■■ yara-rules/ ← Custom YARA rule files (.yar)
■■■ venv/ ← Python virtual environment (recommended)

<script_dir>/Good/ ← OUTPUT_DIR — approved ZIP archives
```

Create the structure:

```
sudo mkdir -p /opt/ai-transit/{fetch,quarantine,approved,reports,logs,yara-rules}
sudo chmod 700 /opt/ai-transit/quarantine
sudo chown -R $USER:$USER /opt/ai-transit

# Create output directory next to scripts
mkdir -p ~/ai-transit/Good
```

6. Environment Variables

Variable	Default	Description
`WORK_DIR`	`/opt/ai-transit`	Root working directory
`OUTPUT_DIR`	`<script_dir>/Good`	Output directory for approved ZIPs

Set them in your shell profile (`~/.bashrc` or `~/.zshrc`):

```
export WORK_DIR="/opt/ai-transit"
export OUTPUT_DIR="/opt/ai-transit/approved"
```

Or pass them inline per execution:

```
WORK_DIR=/data/ai-transit OUTPUT_DIR=/data/approved ./ai_transit.sh https://github.com/org/repo
```

7. Verification Checklist

Run this script to verify your installation:

```
#!/usr/bin/env bash
# verify_install.sh — checks all required and optional tools

PASS=0; WARN=0; FAIL=0

check() {
local name="$1" cmd="$2"
if command -v "$cmd" &>/dev/null; then
echo -e "\033[32m[OK]\033[0m $name ($cmd)"
(( PASS++ ))
else
echo -e "\033[33m[MISS]\033[0m $name ($cmd) — optional"
(( WARN++ ))
fi
}

require() {
local name="$1" cmd="$2"
if command -v "$cmd" &>/dev/null; then
echo -e "\033[32m[OK]\033[0m $name ($cmd)"
(( PASS++ ))
else
echo -e "\033[31m[FAIL]\033[0m $name ($cmd) — REQUIRED"
(( FAIL++ ))
fi
}

echo "=== Required tools ==="
require "Bash 5+" bash
require "Git" git
require "Python 3" python3
require "pip" pip
require "curl" curl
require "jq" jq
require "zip" zip
require "sha256sum" sha256sum
require "file (MIME)" file

echo ""
echo "=== Python packages ==="
python3 -c "import openpyxl" 2>/dev/null \
&& echo -e "\033[32m[OK]\033[0m openpyxl" && (( PASS++ )) \
|| { echo -e "\033[31m[FAIL]\033[0m openpyxl — run: pip install openpyxl"; (( FAIL++ )); }

echo ""
echo "=== Security tools — Layer 1 ==="
check "gitleaks" gitleaks
check "detect-secrets" detect-secrets
check "ClamAV" clamscan
check "YARA" yara
```

```
echo ""
echo "=== Security tools — Layer 2 ==="
check "Semgrep" semgrep

echo ""
echo "=== Security tools — Layer 3 ==="
check "trivy" trivy
check "pip-audit" pip-audit
check "safety" safety
check "npm audit" npm

echo ""
echo "=== Security tools — Layer 5 ==="
check "Bandit" bandit
check "ShellCheck" shellcheck
check "cppcheck" cppcheck
check "hadolint" hadolint
check "checkov" checkov

echo ""
echo "=== Summary ==="
echo -e " \033[32mOK: $PASS\033[0m | \033[33mOptional missing: $WARN\033[0m | \033[31mRequired missing: $FAIL\033[0m"
[[ $FAIL -eq 0 ]] && echo -e "\033[32mInstallation OK — pipeline ready\033[0m" \
|| echo -e "\033[31mInstallation INCOMPLETE — fix FAIL items above\033[0m"
```

Run it:

```
chmod +x verify_install.sh && ./verify_install.sh
```

8. Running the Pipeline

8.1 Grant execution permissions

```
chmod +x fetch_repo.sh scan_pipeline.sh ai_transit.sh
```

8.2 Basic usage

```
# Scan a public GitHub repository (default branch)
./ai_transit.sh https://github.com/org/my-ai-project

# Scan a specific branch
./ai_transit.sh https://github.com/org/my-ai-project main

# Scan a local directory
./ai_transit.sh /path/to/local/repo
```

8.3 Expected output (PASS case)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
AI Transit Pipeline — Démarrage
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

[INFO] Source : https://github.com/org/repo
[INFO] Work dir : /opt/ai-transit

■■ Phase 1 : Récupération ■■■■■■■■■■■■■■■■■■■■■■■■
[INFO] Clonage : https://github.com/org/repo → /opt/ai-transit/fetch/repo_20250812_143022
[OK] Métadonnées .git supprimées
[OK] Manifest écrit

■■ Phase 2 : Scan de sécurité ■■■■■■■■■■■■■■■■■■■■
...

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ✔ SCAN RÉUSSI (PASS) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
[OK] Archive créée : ./Good/repo_20250812_143022_20250812_143045.zip
[OK] Rapport Excel inclus : scan_report_20250812_143045.xlsx
```

8.4 Expected output (FAIL case)

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ✘ SCAN ÉCHOUÉ (FAIL) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

[ERREUR] Le dépôt n'a pas passé le scan de sécurité.
[ERREUR] Les fichiers ont été déplacés en quarantaine : /opt/ai-transit/quarantine
[ERREUR] Rapport détaillé : /opt/ai-transit/reports/report_20250812_143210.json
```

9. Security Hardening Recommendations

9.1 Run as a dedicated non-root user

```
# Create a dedicated service account
sudo useradd -r -m -d /opt/ai-transit -s /bin/bash aitransit
sudo chown -R aitransit:aitransit /opt/ai-transit

# Run the pipeline as that user
sudo -u aitransit ./ai_transit.sh https://github.com/org/repo
```

9.2 Network isolation (Linux — recommended for production)

The pipeline only needs outbound HTTPS to `github.com` and `api.github.com`. Restrict all other outbound traffic:

```
# Using ufw
sudo ufw default deny outgoing
sudo ufw allow out 443 comment "HTTPS for GitHub"
sudo ufw allow out 53 comment "DNS"
sudo ufw enable
```

For air-gapped environments, use a forward proxy (e.g., Squid) with a whitelist:

```
acl github_whitelist dstdomain .github.com .githubusercontent.com
http_access allow github_whitelist
http_access deny all
```

9.3 Quarantine directory isolation

The `quarantine/` directory is set to `chmod 700` automatically. For additional isolation, mount it on a separate filesystem with `noexec`:

```
# /etc/fstab entry
tmpfs /opt/ai-transit/quarantine tmpfs rw,noexec,nosuid,nodev,size=2G 0 0
```

9.4 YARA custom rules

Place your organization's YARA rules in `/opt/ai-transit/yara-rules/*.yar`. The pipeline automatically loads all `.yar` files in that directory during the L1 scan.

Example minimal rule:

```
rule Suspicious_Base64_Exec {
meta:
description = "Detects base64-encoded exec calls"
strings:
$b64_exec = /eval\(base64_decode\(/ nocase
condition:
$b64_exec
}
```

9.5 Log rotation

```
# /etc/logrotate.d/ai-transit
/opt/ai-transit/logs/*.log {
daily
rotate 30
compress
missingok
notifempty
}
```

9.6 Regular tool updates

Schedule weekly updates for security tool databases:

```
# /etc/cron.weekly/ai-transit-update
#!/bin/bash
freshclam # ClamAV virus definitions
trivy image --download-db-only # trivy CVE database
semgrep --update # Semgrep rules
```

10. Troubleshooting

Symptom	Likely cause	Fix
`fetch_repo.sh: Hôte refusé`	Non-GitHub URL passed	`Only github.com URLs are accepted`
`Repo trop volumineux`	Repository exceeds 500 MB	`Review the target repo; increase MAXSIZEMB in fetch_repo.sh if justified`
`openpyxl manquant`	Python package not installed	`pip install openpyxl`
Excel file not in ZIP	`openpyxl import fails silently`	`Run python3 generateexcelreport.py manually to see the error`
`declare -A: invalid option`	Bash < 4.0 (e.g., macOS default bash)	`Install Bash 5 via Homebrew (brew install bash) or use WSL2`
Semgrep not finding rules	`Semgrep not logged in / no rules`	`semgrep login or use --config auto offline`
trivy DB not found	First run without internet	`trivy image --download-db-only with internet access first`
`chmod 700 fails on quarantine`	Running as non-owner	`Run as the aitransit service account or with sudo`
`zip: command not found`	zip not installed	`sudo apt-get install zip`
Pipeline always FAILs on WARN	Strict mode not intended	WARNs do not block the pipeline; only FAIL findings block it
Git clone fails on private repos	Private repo URL	Only public GitHub repositories are supported

Quick Reference — One-Line Install (Ubuntu 22.04)

```
# Core + all optional tools in one command (Ubuntu 22.04)
sudo apt-get update && \
sudo apt-get install -y bash git curl jq zip unzip file coreutils \
python3 python3-pip python3-venv shellcheck cppcheck clamav yara nodejs && \
python3 -m venv /opt/ai-transit/venv && \
source /opt/ai-transit/venv/bin/activate && \
pip install openpyxl detect-secrets bandit pip-audit safety semgrep checkov && \
curl -sSL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh | sudo sh -s --
-b /usr/local/bin && \
sudo freshclam && \
```

```
echo "Installation complete"
```

11. Self-Scan: Verifying the Installation is Safe

Before deploying the pipeline in a production or enterprise environment, you should verify that:

The **bundle scripts and Python files** are themselves free of malicious patterns.
The **third-party binaries** you installed match their published checksums.
The **Python packages** you pulled from PyPI do not carry known CVEs.
The **installed tools** have not been tampered with after installation.

This section walks through each of these verification steps.

11.1 Run the pipeline against its own source code (meta-scan)

The most natural way to validate the bundle is to scan it with itself.

```
# Point the pipeline at the directory containing the scripts
./ai_transit.sh /path/to/ai-transit-bundle
```

A clean installation will produce a **PASS** result. Any unexpected FAIL or WARN finding on the bundle's own files should be investigated before deployment.

> ***What this checks:***
>
> *- L1: gitleaks / detect-secrets scan the* `.sh` *and* `.py` *files for accidentally embedded secrets or credentials.*
>
> *- L2: Semgrep applies OWASP / CWE / CERT rules to the Python source (*`generateexcelreport.py`*,* `build_*.py`*).*
>
> *- L4: Universal pattern checks run on every file (hardcoded credentials, path traversal, dangerous calls).*
>
> *- L5: Bandit runs on every* `.py` *file; ShellCheck runs on every* `.sh` *file.*

11.2 Verify binary checksums

For each third-party binary you downloaded manually, compare its SHA-256 hash against the value published on the official release page before first use.

```
#!/usr/bin/env bash
# check_binaries.sh — verify installed binary hashes

PASS=0; FAIL=0

verify() {
local name="$1" path="$2" expected_sha="$3"
if [[ ! -f "$path" ]]; then
echo "[SKIP] $name — not installed"
return
fi
actual=$(sha256sum "$path" | awk '{print $1}')
if [[ "$actual" == "$expected_sha" ]]; then
echo -e "\033[32m[OK]\033[0m $name — checksum matches"
```

```
(( PASS++ ))
else
echo -e "\033[31m[FAIL]\033[0m $name — CHECKSUM MISMATCH"
echo " Expected : $expected_sha"
echo " Got : $actual"
(( FAIL++ ))
fi
}

# Replace the SHA values below with those published on each tool's GitHub release page
# for the exact version you installed.
echo "=== Binary integrity check ==="
verify "gitleaks" "/usr/local/bin/gitleaks" \
"<SHA256_FROM_GITLEAKS_RELEASE_PAGE>"

verify "trivy" "/usr/local/bin/trivy" \
"<SHA256_FROM_TRIVY_RELEASE_PAGE>"

verify "hadolint" "/usr/local/bin/hadolint" \
"<SHA256_FROM_HADOLINT_RELEASE_PAGE>"

echo ""
echo "Result: OK=$PASS FAIL=$FAIL"
[[ $FAIL -eq 0 ]] && echo "All checksums verified." \
|| echo "STOP — do not use tampered binaries."
```

How to get the expected SHA:

- **gitleaks**: `https://github.com/gitleaks/gitleaks/releases` → `checksums.txt` attached to the release.
- **trivy**: `https://github.com/aquasecurity/trivy/releases` → `trivy_<version>_Linux-64bit.tar.gz.sha256`.
- **hadolint**: `https://github.com/hadolint/hadolint/releases` → SHA-256 listed next to the binary download link.

```
chmod +x check_binaries.sh && ./check_binaries.sh
```

11.3 Verify GPG signatures (optional but recommended)

Several tools publish GPG-signed release artifacts. Verifying the signature provides a stronger guarantee than a checksum alone (the checksum file itself could be tampered with).

Example for gitleaks:

```
# Import the gitleaks signing key
curl -sSL https://github.com/gitleaks.gpg | gpg --import

# Download the release signature alongside the binary
GITLEAKS_VERSION="8.18.4"
curl -sSL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz.sig" \
-o /tmp/gitleaks.sig

curl -sSL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz" \
-o /tmp/gitleaks.tar.gz

# Verify
```

```
gpg --verify /tmp/gitleaks.sig /tmp/gitleaks.tar.gz \
&& echo "Signature OK" \
|| echo "SIGNATURE INVALID — do not install"
```

Example for trivy (cosign — supply chain verification):

```
# Install cosign
curl -sSL https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-amd64 \
-o /usr/local/bin/cosign && chmod +x /usr/local/bin/cosign

# Verify trivy binary with cosign transparency log
TRIVY_VERSION=$(trivy --version | head -1 | awk '{print $2}')
cosign verify-blob \
--certificate "https://github.com/aquasecurity/trivy/releases/download/v${TRIVY_VERSION}/trivy_${TRIVY_VERSION}_Linux-64bit.tar.gz.pem" \
--signature "https://github.com/aquasecurity/trivy/releases/download/v${TRIVY_VERSION}/trivy_${TRIVY_VERSION}_Linux-64bit.tar.gz.sig" \
--certificate-identity-regexp "https://github.com/aquasecurity/trivy" \
--certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
/usr/local/bin/trivy \
&& echo "trivy supply-chain signature OK"
```

11.4 Audit Python dependencies (PyPI supply-chain check)

The Python packages installed for the pipeline should be scanned for known CVEs and typosquatting risks.

```
# Activate the virtual environment first
source /opt/ai-transit/venv/bin/activate

# Generate a requirements file from the installed packages
pip freeze > /tmp/ai-transit-requirements.txt

# CVE scan with pip-audit
pip-audit -r /tmp/ai-transit-requirements.txt

# Advisory scan with safety
safety check -r /tmp/ai-transit-requirements.txt

# Optional: use trivy to scan the venv as a filesystem target
trivy fs /opt/ai-transit/venv \
--scanners vuln \
--severity HIGH,CRITICAL \
--exit-code 1
```

A clean environment will return exit code 0 with no findings. Any HIGH or CRITICAL CVE in `openpyxl`, `semgrep`, `bandit`, `detect-secrets`, `pip-audit`, or `safety` itself should be remediated by upgrading the affected package before using the pipeline.

```
pip install --upgrade openpyxl semgrep bandit detect-secrets pip-audit safety checkov
```

11.5 Scan installed system packages for CVEs

```
# Use trivy to audit the OS package list
trivy rootfs / \
--scanners vuln \
--severity HIGH,CRITICAL \
--ignore-unfixed \
--output /opt/ai-transit/reports/host_cve_report.json \
--format json

# Human-readable summary
trivy rootfs / \
--scanners vuln \
--severity HIGH,CRITICAL \
--ignore-unfixed
```

> *This does not require root.* `trivy` *reads* `/var/lib/dpkg/status` *(Debian/Ubuntu) or* `/var/lib/rpm/Packages` *(RHEL/CentOS) to enumerate installed packages and cross-references them with the NVD/OSV/GitHub Advisory databases.*

11.6 Detect post-install tampering (AIDE / file integrity)

For production environments, configure a file integrity monitor on the pipeline scripts to detect any modification after installation.

```
# Install AIDE (Advanced Intrusion Detection Environment)
sudo apt-get install -y aide

# Initialize the AIDE database covering only the pipeline directory
sudo aide --config=/etc/aide/aide.conf --init \
--rule "Dir=/path/to/ai-transit-bundle p+i+n+u+g+s+b+md5+sha256"

sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Run a check at any time (e.g. from cron or before each pipeline run)
sudo aide --check
```

A simpler alternative — store SHA-256 hashes of all bundle files at install time and re-verify before each run:

```
#!/usr/bin/env bash
# generate_bundle_manifest.sh — run once at install time
BUNDLE_DIR="$(dirname "$0")"
sha256sum "${BUNDLE_DIR}"/*.sh "${BUNDLE_DIR}"/*.py > "${BUNDLE_DIR}/.bundle_manifest.sha256"
echo "Manifest saved to .bundle_manifest.sha256"

# verify_bundle.sh — run before each pipeline execution
sha256sum --check "${BUNDLE_DIR}/.bundle_manifest.sha256" \
&& echo "Bundle integrity OK" \
|| { echo "TAMPERING DETECTED — do not run the pipeline"; exit 1; }
```

Add the integrity check at the top of `ai_transit.sh` to make it automatic:

```
# Add to ai_transit.sh, before the first phase
```

```
if [[ -f "${SCRIPT_DIR}/.bundle_manifest.sha256" ]]; then
sha256sum --check --quiet "${SCRIPT_DIR}/.bundle_manifest.sha256" \
|| die "Bundle integrity check failed — scripts may have been tampered with."
fi
```

11.7 Summary — self-scan checklist

Step	Command / Tool	Expected result
Meta-scan of bundle	`./ai_transit.sh /path/to/bundle`	PASS
Binary checksums	`./check_binaries.sh`	All OK, no FAIL
`GPG / cosign signatures`	`gpg --verify / cosign verify-blob`	Signature valid
Python CVE scan	`pip-audit + safety check`	`No HIGH/CRITICAL`
Python trivy scan	`trivy fs /opt/ai-transit/venv`	Exit code 0
Host OS CVE scan	`trivy rootfs /`	`No unpatched HIGH/CRITICAL`
Bundle integrity	`sha256sum --check .bundle_manifest.sha256`	OK
AIDE check (production)	`sudo aide --check`	No modifications

> ***Recommended practice:*** *run steps 1, 4, and 7 automatically before each pipeline execution by wrapping them in a pre-flight script called by* `ai_transit.sh`*.*

AI Transit Pipeline — Installation Guide v2.0 — 2025